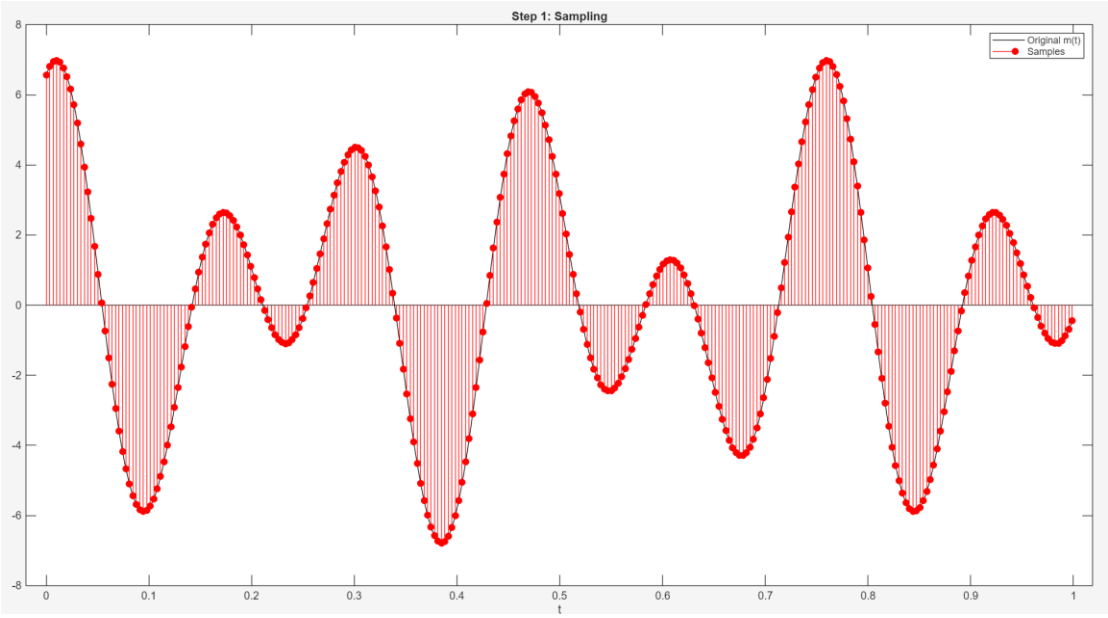
HW2

112511071 EE 楊文德

▼ Sample m(t) with 300Hz

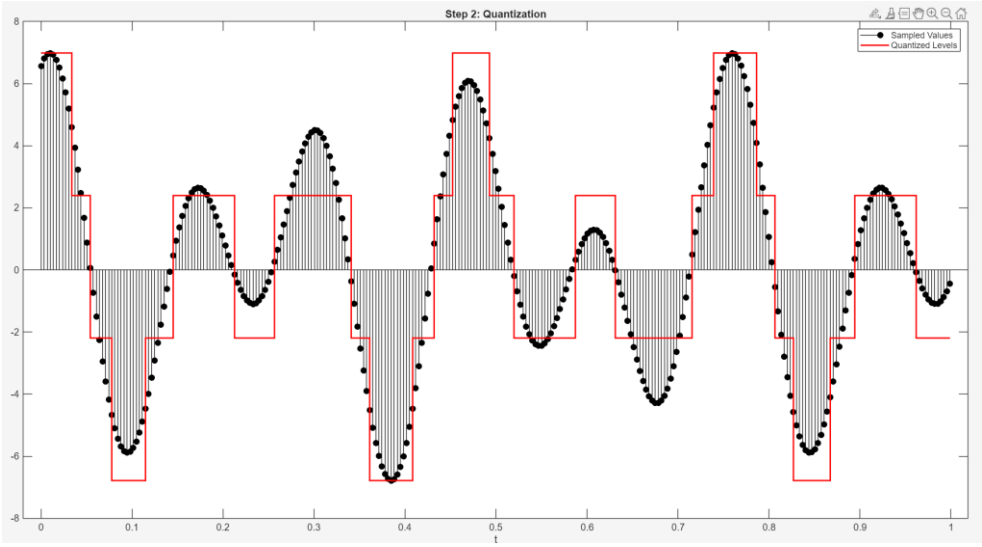


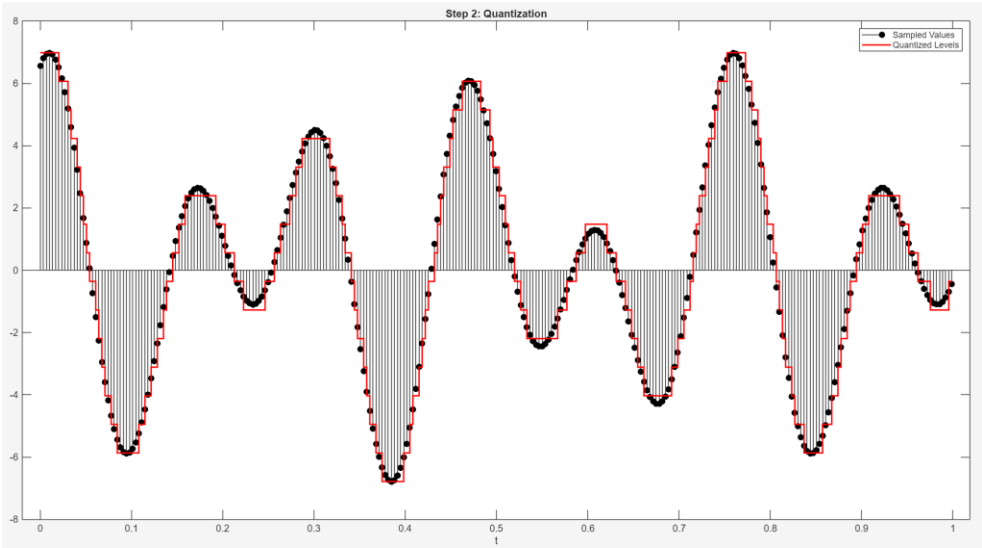
Code:

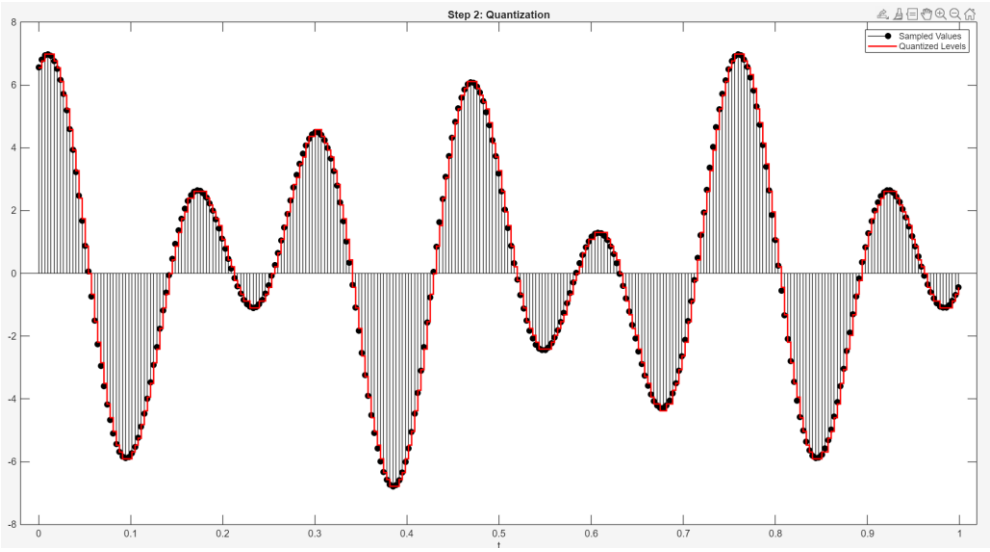
```
%% ---------- 1. Sampling ----------
n = 1:Ns:length(m);     % sampling period is Ns
m_s = m(n);
```

在這裡我們要對原本的連續訊號做取樣，但我們現在是用電腦模擬的關係所以要用數經過幾個離散點的方式，而這程式 $round(\frac{fd}{fs})$ 就是在計算取樣等於是取幾個離散點，以達到 $m[n] = m(nT)$ 的效果。在 matlab 當中我們用向量 = 起始值 ：步長（間隔）：終止值 來刻劃取樣點。

▼ Quantization sampling (L = 4、16、64)







Code:

```
%% ---------- 2. Quantization ----------
m_min = min(m_s);
m_max = max(m_s);
delta = (m_max-m_min)/(L-1); %quantization interval
            %devide interval [m_min, m_max] into L-1 part

q_index = round((m_s-m_min)/delta);  %quantization index, each value 0~L-1
               %try the function "round"
a = m_min+delta*q_index;  %quantization value
```

為了 quantize sampling，我們要先決定具體是要分幾階(L)，然後再根據階層數將 [min,max] 之間的範圍分成 L 階，每個階層之間的間距定義成 delta

$$\delta = \frac{m_{max} - m_{min}}{L - 1}$$

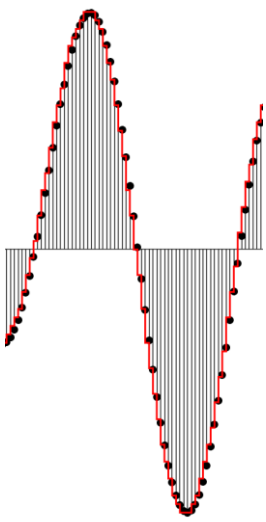
然後根據這 delta 我們將剛剛 sampling 的波轉換成階層數(q_index)，也就是

$$q_{index} = round\left(\frac{(m_s - m_{min})}{\delta}\right)$$

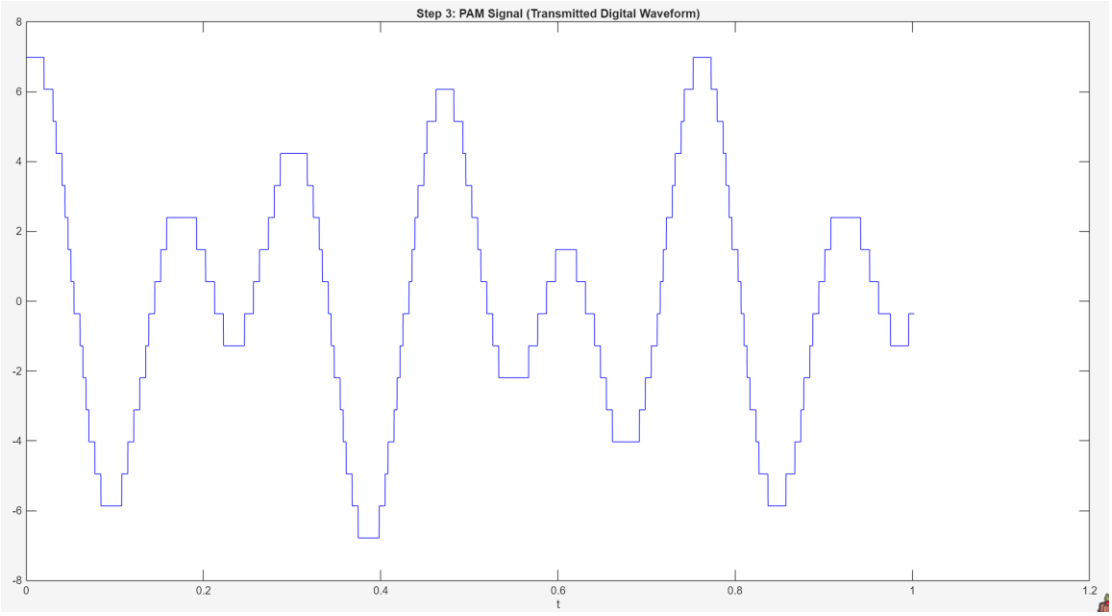
因為我們將 m_min 當作基準點，所以我們在 quantization 時，要先垂直位移 m_min 量之後再除以 delta，最後四捨五入就大功告成，不過這樣得知道的是階層數(0~L-1)，要轉回電訊號的話是需要以下的步驟:

$$Level = m_{min} + \delta * q_{index}$$

然後根據 L 值從 4、16、64 切分得越來越細，我們可以觀察到越來越貼近 sampling 的波型，甚至到 L=64 時已經和 sampling 300Hz 的效果差不多了。



▼ PAM signal



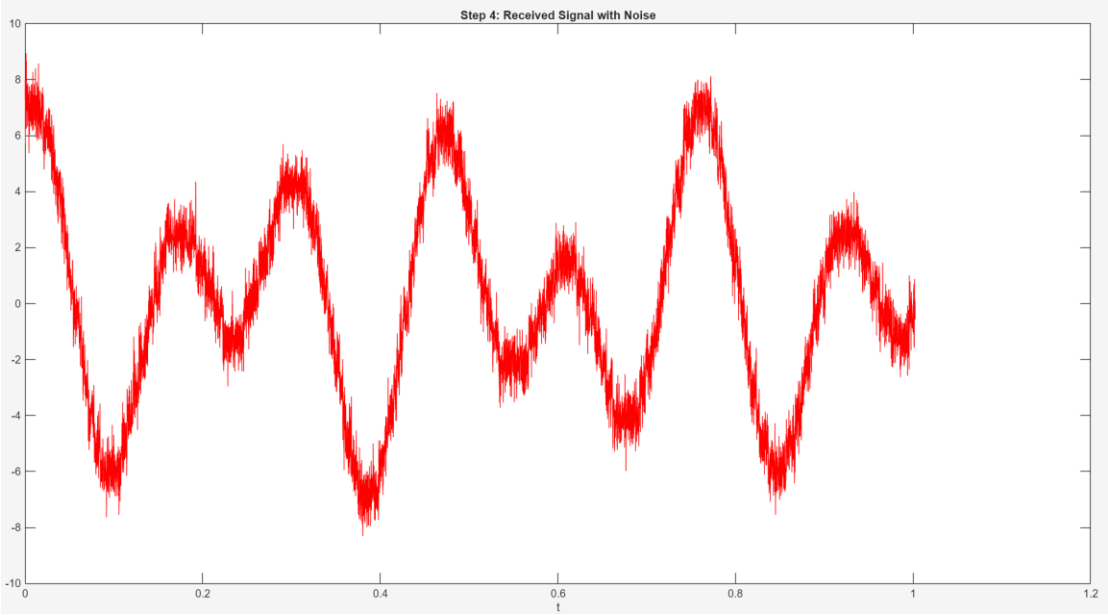
在這裡我們實現了脈衝振幅調變 (PAM)，其物理意義等同於發射端的數位類比轉換 (DAC) 與零階保持 (ZOH) 行為。我們將 Step 2 得到的離散量化振幅 a_n，乘上一個寬度為週期 T_s 的矩形脈衝 p(t)。如以下公式所示，各個脈衝在時域上平移與疊加後，即可建構出適合在物理通道中傳輸的連續時間類比基頻波形 s(t)：

$$s(t) = \sum_{n=-\infty}^{\infty} a_n p(t - nT_s)$$

Code:

```
for k = 1:length(a)
    s_pam((k-1)*Ns+1:k*Ns) = a(k)*pulse;
end
```

▼ S_pam add noise



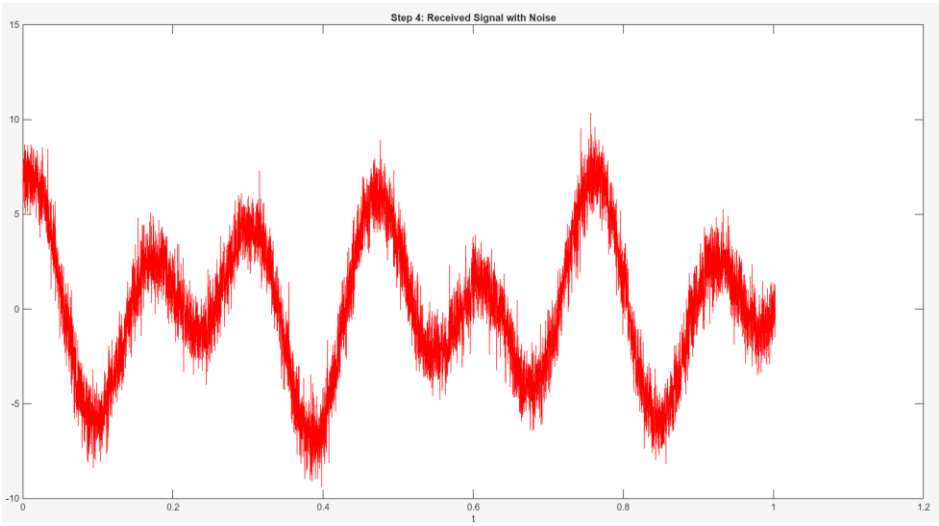
我們在上一部份產生了 PAM 的訊號，但是在現實世界當中訊號往往都會被各種電磁干擾、熱干擾等等因素而變得有很多毛刺，在這裡我們用 gauss distribution 模擬。

Code:

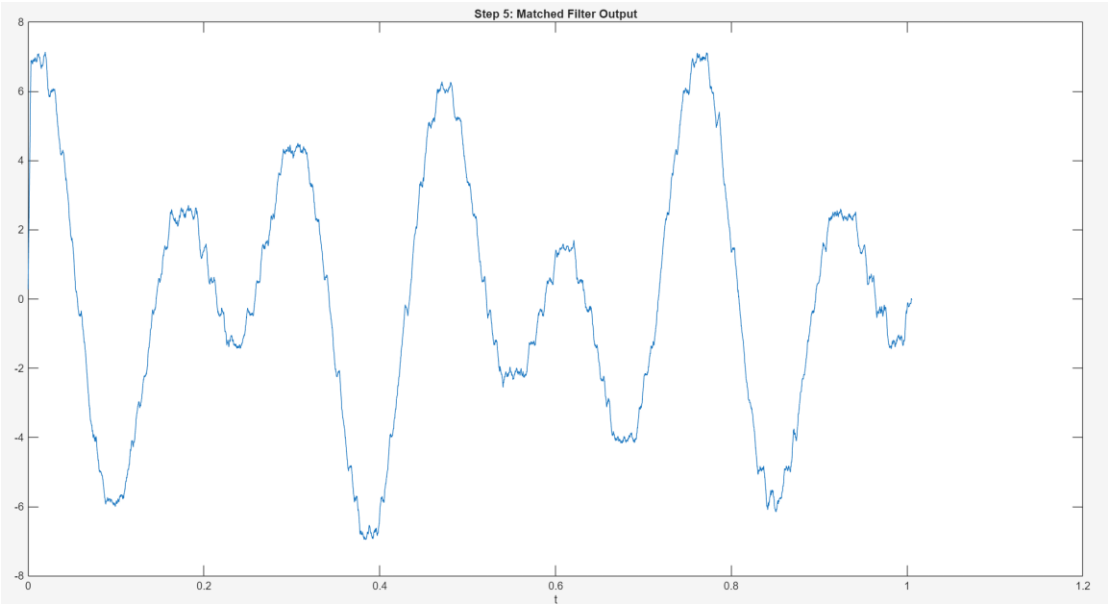
```
r_pam = s_pam + sqrt(noise_power)*randn(size(s_pam));
```

MATLAB 的 randn 函數會產生一個與 s_pam 長度相同，且符合標準常態分布的隨機序列。標準常態分布的平均值為 0，變異數為 1，即$X \sim \mathcal{N}(0,1)$。

當 noise_power 越大訊號越容易受到干擾:



▼ R_pam conv. filter



Matched Filter 的脈衝響應 h(t) 是一 pulse，也就是矩形波，在時域上，兩個矩形波的卷積會產生三角波，將原本毛刺的階梯邊緣變的平滑；在頻域上，矩形窗等效於 Sinc 函數，可以將其視為 Low-Pass Filter 的特性。因此，我們可以發現到原本粗糙越發有 quantization 的輪廓。

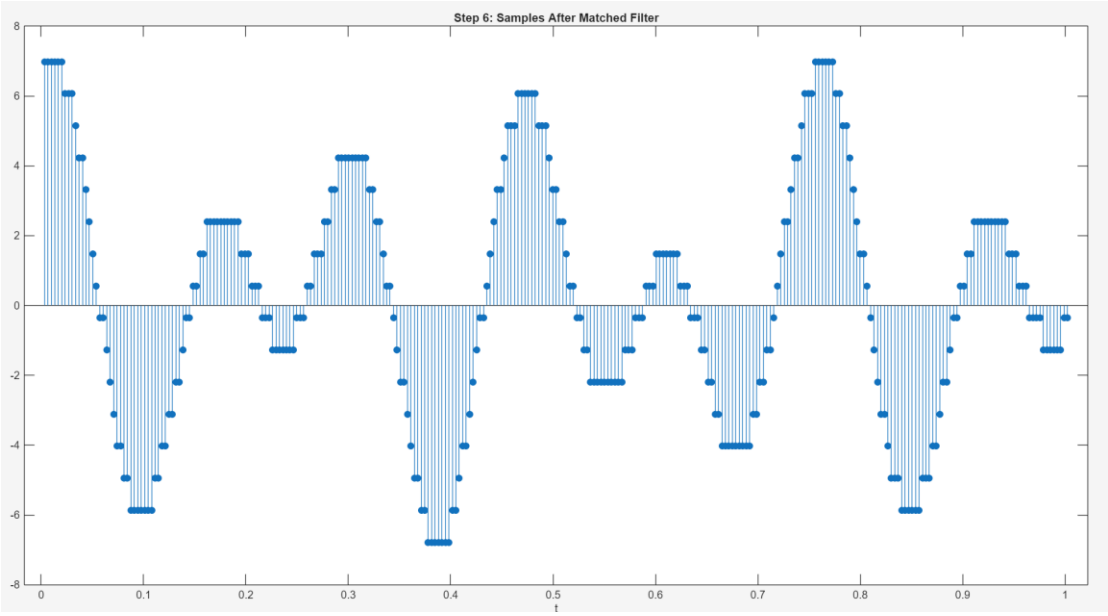
Code:

```
y = conv(r_pam,h);  %pass the signal r_pam through filter h
                    %try function "conv"
y = y / sum(pulse.^2);
t_y = (0:length(y)-1)/fd;
```

在這裡我很好奇為什麼要有這兩個步驟?後來回去翻通原(一)的講義，發現到以下兩點性質可以解釋為甚麼會這樣寫:

1. 因為 match filter 基本上是對訊號做積分，這會讓波型會被放大，而增益剛好會等於矩形波的能量，因此我們消除這方面的影響。
2. 由於卷積運算會使離散陣列的長度擴張(N1+N2-1，N1、N2 各為 convolution 的兩訊號寬度)，為了在實體物理世界中正確呈現波形，我們必須將純粹的數位陣列索引 (Index)，除以系統的模擬取樣率 f_d

▼ Sample signal



Code:

```
sample_index = Ns:Ns:length(y);
sample_points = y(sample_index);
```

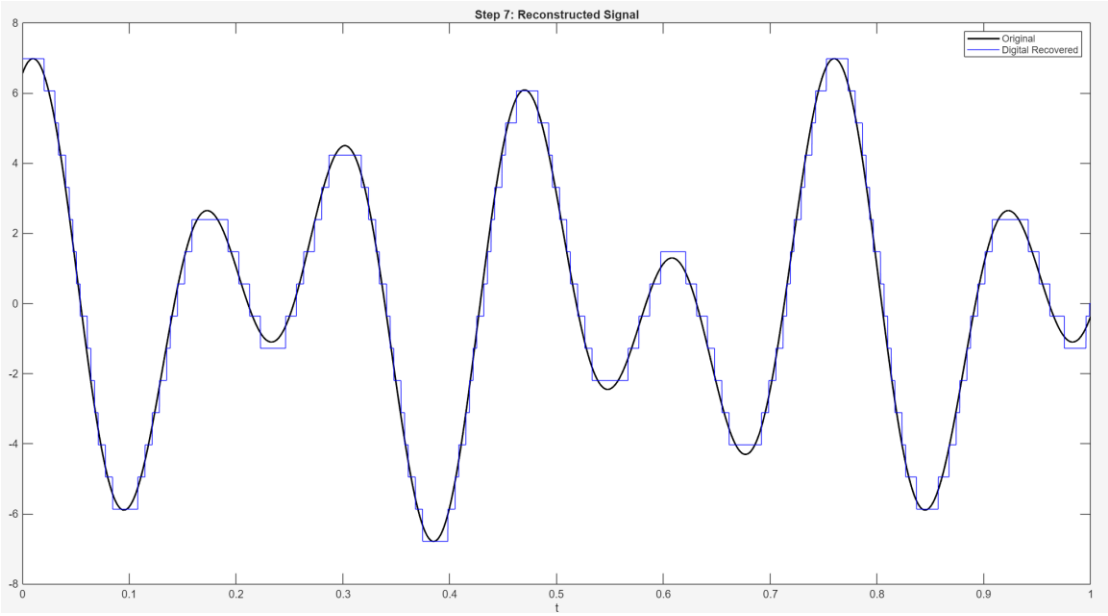
為了要 recover 訊號，我們首先要繼續對訊號做 sampling，只取有參考價值的離散點(sample_point(sample_index))即可。

Code:

```
for k = 1:length(a)
    a_hat(k) = round((sample_points(k)-m_min)/delta)*delta+m_min;
                %a_hat(k) is the closest quatization level
                %to the sample point(k)
                %%try the function "round"
end
```

Sampling 完後之後我們要繼續之前 step.2 的步驟，將訊號四捨五入找到最近的 quantize level 然後在量化成電訊號的 level。

▼ Reconstructed signal with square wave



Code:

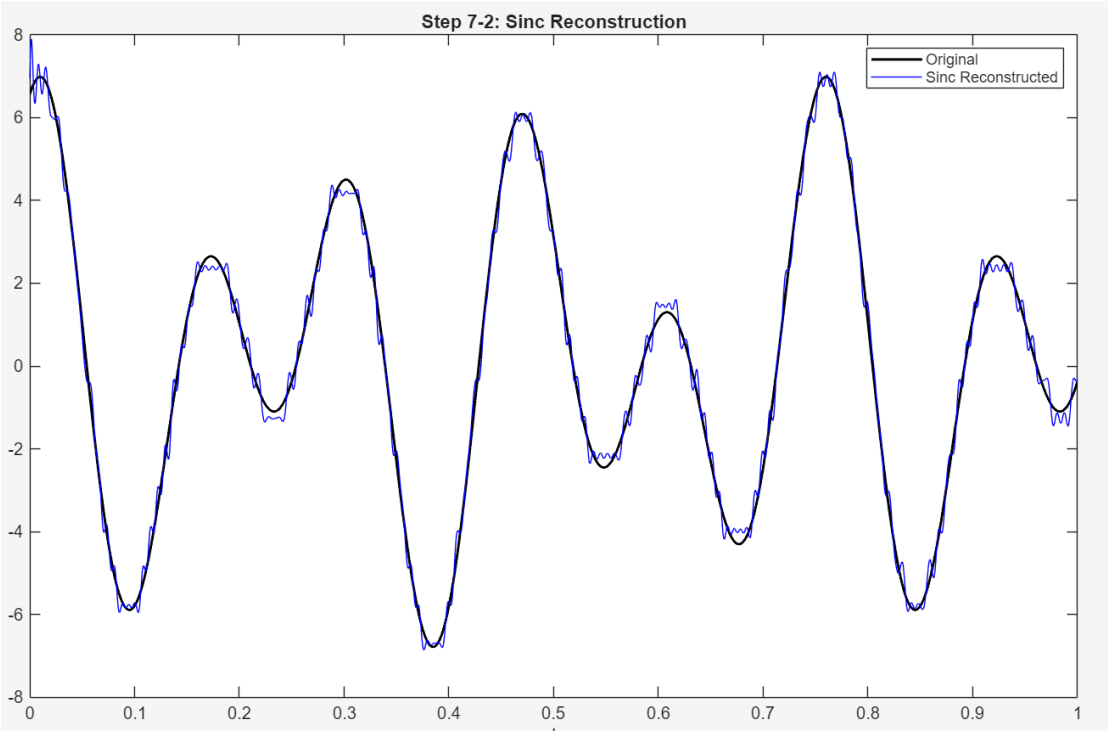
```
for k = 1:length(a_hat)
    idx = (k-1)*Ns+1:k*Ns;
    if idx(end) <= length(m_rec_dig)
        m_rec_dig(idx) = a_hat(k);
    end
end
```

這部分基本上就是將之前所測得的 quantization level 延續一個週期，與 step.3 PAM 訊號的部分有異曲同工之妙。

為了要防止 index 超出 m_rec_dig 的範圍而導致 memory 錯誤，所以我們要在對 m_reg_dig 賦值前對 index 做末位檢測，以下就是此條件的寫法:

```
if idx(end) <= length(m_rec_dig)
```

▼ Reconstructed signal with sinc wave



Code:

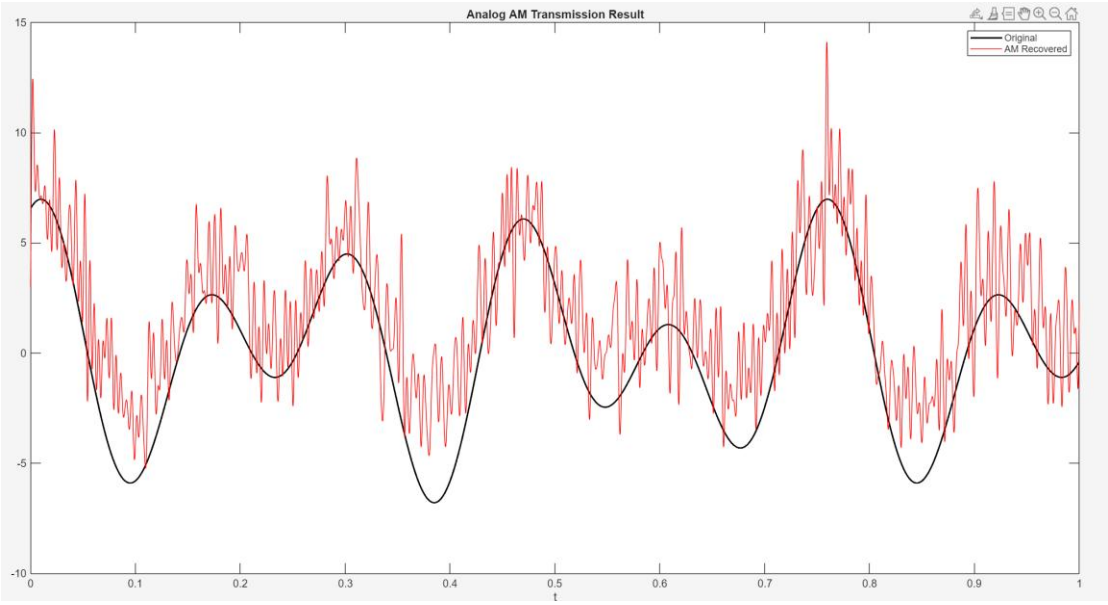
```
for k = 1:length(a_hat)
    m_rec_sinc = m_rec_sinc + a_hat(k) * sinc((t - t_sample(k)) * fs);
end
```

這部分與之前的 PAM signal 類似，不過這次是用 sinc 函數模擬而成，根據助教的提示:

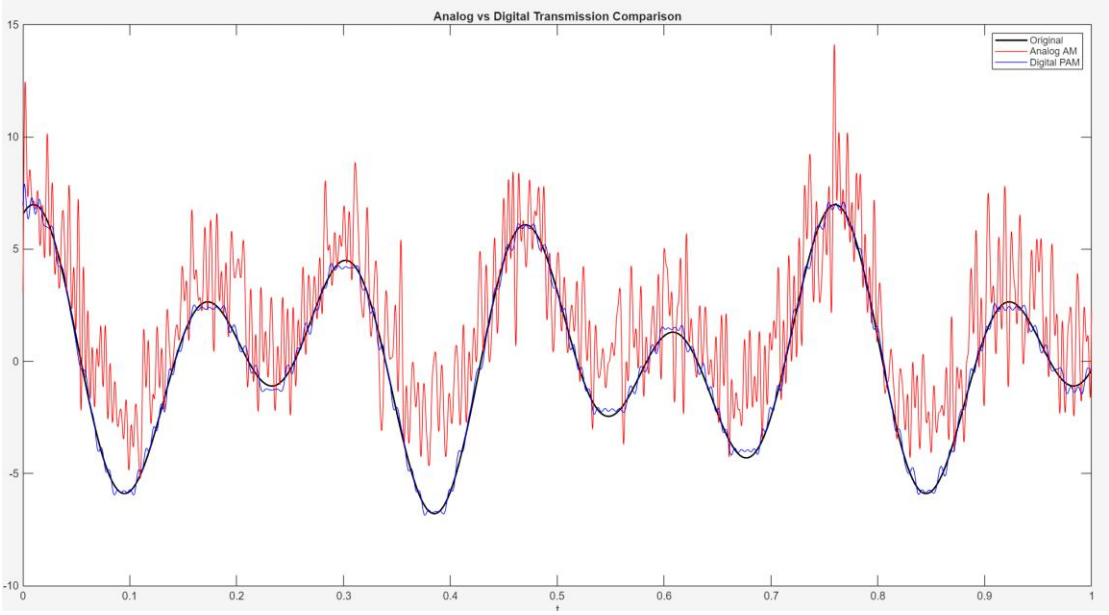
$$\widehat{m}(t) = \sum_n \hat{a}_n \mathrm{sinc}\left(\frac{t - nT_s}{T_s}\right)$$

因為 sinc 函數是無限延伸，而不是除了週期外都是零的 pulse function，所以比 PAM 多了 m_rec_sinc 的累加單元。而之所以會用 sinc 函數的原因我覺得跟之前提到的類似，這樣就可以規避掉高頻的成分汙染波型。

▼ AM recover signal



▼ Analog vs digital modulation



從兩張圖我們可以發現到數位的調變與解調效果會比類比來的好很多，我認為之所以會造成這現象的原因有以下幾個:

1. Match filter 的積分效應 ：

   在取樣之前，數位系統讓訊號通過了 match filter。除此之外 convolution 的的運算能將一個符號週期內的雜訊正負抵消，讓目標訊號能量線性疊加，確保了在取樣瞬間的訊號雜訊比 (SNR) 達到理論最大值。

2. 離散量化：

因為數位系統的合法電壓只有特定的幾個離散階層。當帶有微小雜訊的訊號進入接收端進行 quantization 時，只要雜訊的擾動幅度沒有越過相鄰階層距離的一半（即 delta/2），系統就能透過 round() 函數進行矯正，將偏離的浮點數值設定回原本的 level，這很大程度上避免 noise 的干擾。